Программная инженерия

Лекция 5: Контейнеризация и виртуализация. Docker

Денисов Артём Владимирович

Уральский Федеральный Университет

2026

Проблема "It works on my machine"

Код работает на машине разработчика, но ведёт себя совершенно по-другому в продакшене

- Разные версии библиотек
- Несовместимые зависимости
- Различные ОС
- Отличающиеся окружения

Проблема - традиционный деплой

Backend, frontend, nginx, redis, postgres, ...

- Конфликты окружения
- Отсутствие изолированности
- Сложный деплой

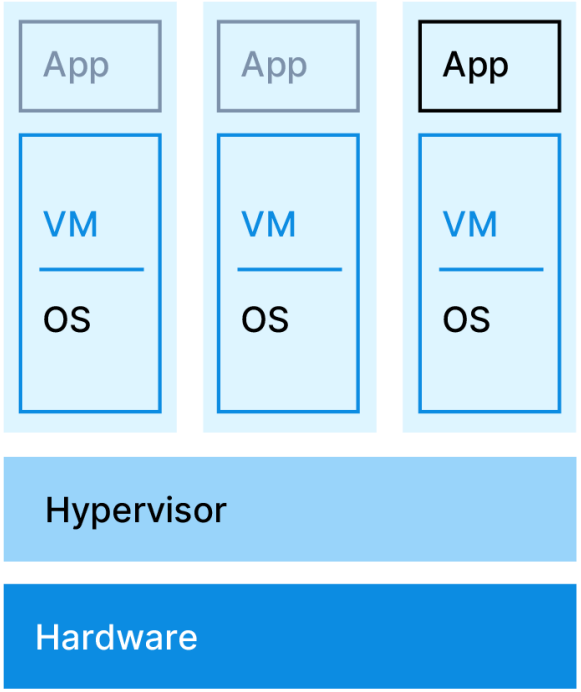
Решение

Для каждой программы создать изолированное окружение

Как это сделать?

- Виртуализация
- Контейнеризация

Виртуализация



Виртуализация создаёт полную копию компьютера внутри реального

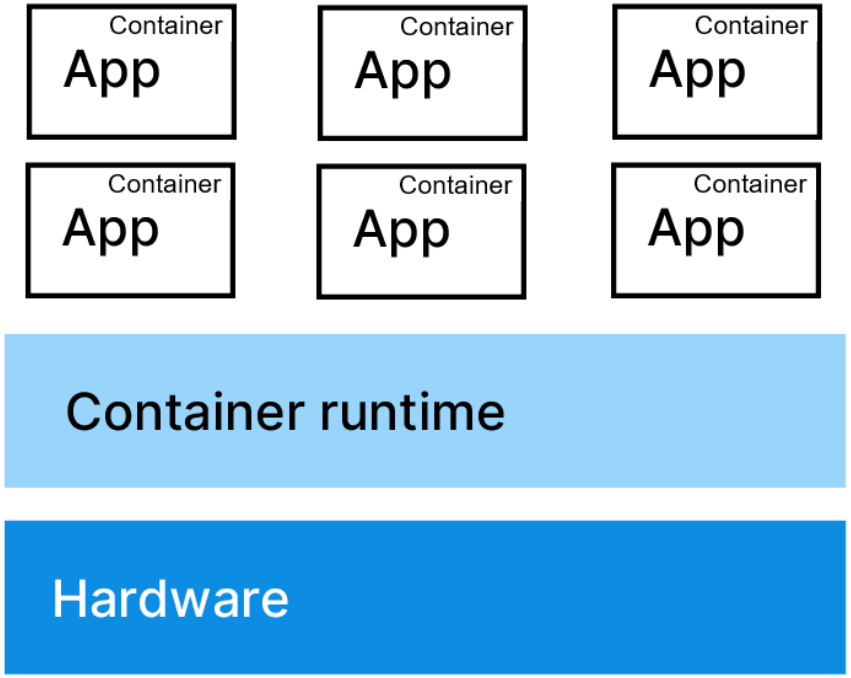
- Каждая ВМ - как «отдельный» компьютер
- Отдельная полноценная ОС
- Гипервизор создаёт и управляет ВМ

Инструменты для виртуализации

Virtualbox, WMVare - гипервизоры

Vagrant - DevOps инструмент для настройки окружения в ВМ

Контейнеризация



Контейнеризация упаковывает приложение с его окружением в лёгкий «контейнер»

- Контейнеры используют ядро ОС хоста
- Контейнеры **изолированы**
- Быстрее и экономичнее VM

Инструменты для контейнеризации

Docker, podman - движки контейнеров

kubernetes

Kubernetes - инструмент оркестрации контейнеров

Docker

Docker это

Платформа для разработки, хранения и запуска приложений внутри легковесных контейнеров.

Термины

- **Docker Image (образ)** - неизменяемый шаблон, содержащий всё необходимое для запуска приложения: код, библиотеки, зависимости, конфигурацию
- **Docker Container (контейнер)** - запущенный экземпляр образа, изолированная среда выполнения приложения

Аналогия с ООП

Image *- класс,* ***Container*** *- объект класса*

Docker контейнер

Свойства контейнера

- Легковесность
- Атомарность
- Изоляция
- один контейнер - одна задача

Docker контейнер содержит

- Код приложения
- Библиотеки и зависимости
- Конфигурационные файлы
- Среду выполнения
- Системные инструменты и библиотеки

Демонстрация Docker

Изучим механики Docker

- Dockerfile команды
- Docker cli команды
- Docker репозитории
- Сборка и запуск Docker Image

Кеширование и слои

- Образ состоит из слоёв; каждая инструкция Dockerfile создаёт новый слой.
- При сборке Docker переиспользует неизменившиеся слои и сильно ускоряет повторные сборки.
- Изменение раннего слоя инвалидирует кеш всех последующих слоёв.

Port forwarding

- Порт контейнера не доступен снаружи, пока мы явно не опубликуем его.
- docker run -p HOST_PORT:CONTAINER_PORT публикует порт контейнера на хосте.
- Можно указывать IP (127.0.0.1 только локально) и протокол tcp/udp в -p.
- Для общения контейнеров между собой часто достаточно одной Docker-сети без -p.

Docker Volumes

- Volume нужен, чтобы данные жили дольше контейнера (логи, БД, загрузки и т.п.).
- Bind mount: явный путь на хосте связывается с путём в контейнере; удобно в разработке.
- Named volume: место хранит Docker, можно легко переносить между хостами; лучше для продакшна.
- Монтируем через -v или более явный синтаксис −mount в docker run / docker compose.

Multistage building

- Один Dockerfile с несколькими FROM разделяет стадии build и runtime.
- В build-стадии ставим тяжёлые инструменты и собираем артефакты.
- В финальный образ копируем только нужные бинарники/статику, без toolchain'ов и исходников.
- Итог: меньший размер, выше безопасность и быстрее CI/CD за счёт кеша между стадиями.

Dockerfile best practices

- Выбирайте минимальные базовые образы (alpine, slim, distroless) там, где это возможно.
- Фиксируйте версии базовых образов и зависимостей для воспроизводимых сборок.
- Сначала устанавливайте зависимости, потом копируйте код приложения — так кеш слоёв работает эффективнее.
- Внутри RUN очищайте временные файлы и кеши, чтобы не раздувать слои.
- Запускайте приложение не от root: создавайте отдельного непривилегированного пользователя.

Виртуализация vs Контейнеризация

- Виртуальная машина виртуализирует целую машину: каждому гостевому ОС нужны свои ресурсы и гипервизор.
- Контейнеры разделяют ядро хоста и изолируют только процессы и файловую систему приложения.
- ВМ дают более сильную изоляцию, но тяжелее и дольше стартуют; контейнеры легче и запускаются за миллисекунды.
- Типичный паттерн: несколько контейнеров внутри одной ВМ в облаке — баланс изоляции и плотности.

Зачем нужен Docker Desktop?

- Упрощённая установка Docker на Windows/macOS: одна установка даёт полный dev-стек.
- Запускает Linux-VM или WSL2 и проксирует docker-команды, скрывая детали виртуализации.
- Удобен разработчикам, которые не хотят сами администрировать Linux-хост для Docker.

Источники информации и изображений

1. https://www.youtube.com/watch?v=AY1mE8JILJQ
2. https://www.youtube.com/watch?v=Ud7Npgi6x8E&list=PLsz00TDipIfcc6X5TECsuk0YNGWIx5HMl&index=3
3. https://documentation.suse.com/smart/virtualization-cloud/html/virtualization/index.html
4. https://github.com/aguyonp/vagrant-glpi
5. https://www.netscaler.com/articles/what-is-containerization
6. https://keploy.io/blog/community/podman-vs-docker

Программная инженерия

Лекция 5: Контейнеризация и виртуализация. Docker

Денисов Артём Владимирович

Уральский Федеральный Университет

2026